Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Seguridad del Software, IC8071

Proyecto #1: Identificación de debilidades mediante análisis estático de código C/C++ v. 6.0

Estudiantes:
Luis Urbina Salazar, 2023156802
Víctor Aymerich Quesada – 2023152436

Segundo semestre del año 2025

Índice general

1. Información del Software **2**
1.1. Nombre y versión del software 2
1.2. Sistema operativo utilizado 2
1.3. Obtención del código fuente 2

2. Instalación del Software y Herramientas **4**
2.1. Instalación de BusyBox 4
2.2. Instalación de Herramientas de Análisis 4
2.3. Ejecución de las Herramientas 5

3. Análisis Automatizado **6**
3.1. Flawfinder 6
3.2. Cppcheck 6
3.3. Clang Static Analyzer 7
3.4. Frama-C (EVA + RTE) 8
3.5. Tabla: Análisis de Hallazgos sobre `httpd.c` (2878 LoC) con LLMs 9
3.6. Verificación de Posibles Falsos Negativos (FN) sobre `httpd.c` . 13
3.7. Discusión sobre el análisis estático 15
3.8. Recomendaciones (basadas en la experiencia) 16

4. Referencias **18**
4.1. Enumeración de CWE 18
4.2. Herramientas de análisis 19

Capítulo 1

Información del Software

1.1. Nombre y versión del software

Para este proyecto se analizó el archivo `networking/httpd.c` del proyecto **BusyBox** en su versión **1.37.0**. BusyBox empaqueta utilidades tipo Unix en un único ejecutable y es ampliamente usado en sistemas embebidos y distribuciones ligeras. El componente `httpd.c` implementa un servidor web ligero (HTTP/1.0) con soporte para CGI y autenticación básica, por lo que resulta un objetivo relevante para evaluación de seguridad.

1.2. Sistema operativo utilizado

El análisis se realizó en **Ubuntu 24.04 LTS (64 bits)** con toolchain estándar de GNU/Linux. Este entorno provee paquetes recientes de compiladores y librerías, además de acceso directo a los fuentes de BusyBox mediante los repositorios oficiales, lo que facilita la reproducibilidad del estudio.

1.3. Obtención del código fuente

El código fuente se obtuvo desde los repositorios de Ubuntu (método recomendado para reproducibilidad). Los pasos seguidos fueron:

```
sudo apt update
sudo apt install build-essential dpkg-dev
apt source busybox
# El  rbol  queda en un directorio similar a: busybox
   -1.37.0/
```

Listing 1.1: Descarga de fuentes de BusyBox 1.37.0 desde repositorios de Ubuntu

Para homogeneizar rutas en el proyecto, se trabajó con la siguiente estructura local (utilizada por las herramientas en secciones posteriores):

```
mkdir -p busybox-source
cp -a busybox-1.37.0 busybox-source/
# Ruta de inter s para el an lisis:
# busybox-source/busybox-1.37.0/networking/httpd.c
```

Listing 1.2: Estructura local de trabajo

Capítulo 2

Instalación del Software y Herramientas

2.1. Instalación de BusyBox

Para obtener el código fuente de **BusyBox 1.37.0**, se utilizó el sistema de paquetes de Debian/Ubuntu. El siguiente comando descarga la versión estable desde los repositorios oficiales y genera la carpeta con el código:

```
sudo apt-get update
apt-get source busybox
# genera: busybox-1.37.0/
```

Listing 2.1: Descarga del código fuente de BusyBox

Posteriormente, se verificó la ubicación del archivo objetivo a analizar:

```
wc -l busybox-1.37.0/networking/httpd.c
```

Listing 2.2: Verificación de archivo httpd.c

Esto confirma que el archivo `httpd.c` cuenta con 2878 líneas de código (aproximadamente).

2.2. Instalación de Herramientas de Análisis

Se emplearon cuatro herramientas libres de análisis estático de seguridad: `flawfinder`, `cppcheck`, `clang static analyzer` y `frama-c`.

```
sudo apt-get update
sudo apt-get install -y flawfinder cppcheck clang frama-
   c-base build-essential
```

Listing 2.3: Instalación de herramientas de análisis estático

Con este procedimiento se aseguraron las versiones disponibles en la distribución Ubuntu 24.04 LTS.

2.3. Ejecución de las Herramientas

Cada herramienta se ejecutó directamente sobre el archivo objetivo `httpd.c`, siguiendo las configuraciones recomendadas para maximizar la cobertura y minimizar falsos positivos:

- **Flawfinder**: inspección de funciones peligrosas.
- **Cppcheck**: análisis con preprocesado y habilitación de todas las categorías de chequeo.
- **Clang Static Analyzer**: detección de vulnerabilidades clásicas de C (uso de punteros nulos, variables no inicializadas, desbordamientos).
- **Frama-C**: análisis formal con `eva` y generación de reportes sobre punteros, memoria y buffers.

Los comandos exactos de ejecución se documentan en la siguiente sección de análisis automatizado.

Capítulo 3

Análisis Automatizado

3.1. Flawfinder

Comando ejecutado

```
flawfinder --columns --minlevel=1 busybox-source/busybox
   -1.37.0/networking/httpd.c \
  > analysis_results/flawfinder_httpd_detailed.txt
```

Justificación de parámetros

- `--columns`: incluye número de línea y columna para ubicar rápidamente el contexto de cada hallazgo.
- `--minlevel=1`: reporta desde el nivel más bajo (1) hasta el más alto (5) para un barrido completo; luego se clasifica manualmente VP/FP.
- Redirección a archivo: asegura trazabilidad y facilita incorporar resultados a la documentación.

3.2. Cppcheck

Comandos ejecutados (dos pasos)

1) Preprocesado con GCC

```
gcc -E -I./include -I./libbb -include include/autoconf.h \
    -D_GNU_SOURCE -DNDEBUG networking/httpd.c > httpd_preprocessed.c
```

2) Ejecución de Cppcheck sobre el archivo preprocesado

```
cppcheck httpd_preprocessed.c --enable=all --xml 2> httpd_clean_analysis.xml
```

Justificación de parámetros

- `-E`: detiene en preprocesado (expande includes y macros) para reducir falsos positivos en proyectos grandes como BusyBox.
- `-I./include -I./libbb` y `-include include/autoconf.h`: replican el entorno real de compilación de BusyBox para que el análisis sea consistente.
- `--enable=all`: habilita todas las categorías (style, performance, portability, information, warning, error) para máxima cobertura.
- `--xml` con redirección de `stderr`: produce un reporte estructurado (`.xml`) apto para post-procesamiento y trazabilidad.

3.3. Clang Static Analyzer

Comando ejecutado:

```
clang --analyze \
  -Ibusybox-source/busybox-1.37.0/include \
  -Ibusybox-source/busybox-1.37.0 \
  -include busybox-source/busybox-1.37.0/include/autoconf_sast.h \
  -D_GNU_SOURCE -DBUSYBOX \
  networking/httpd.c \
  -o analysis_results/clang/httpd_sast.plist \
  2> analysis_results/clang/static.txt
```

Justificación de parámetros:

- `-I` y `-include`: reproducen el entorno de compilación real de BusyBox, garantizando que macros e includes estén disponibles.

- `-D_GNU_SOURCE -DBUSYBOX`: definen banderas requeridas por el código de BusyBox.
- `--analyze`: activa el motor de análisis estático de Clang con los *checkers* de seguridad y de núcleo habilitados.

Resultados destacados:

- Reportes de posibles **CWE-120** (*buffer overflows*) por llamadas a `sprintf` y `strcpy`.
- Detección de accesos a memoria no inicializada (**CWE-457**) en estructuras como `toCgi.wr`.
- Alertas de desreferencia de punteros nulos (**CWE-476**) y posibles fugas de memoria (**CWE-401**).

—

3.4. Frama-C (EVA + RTE)

Comando ejecutado:

```
frama-c -eva -rte \
  -cpp-command "gcc -E -Ibusybox-source/busybox-1.37.0/
     include \
  -D_GNU_SOURCE -DBUSYBOX" \
  busybox-source/busybox-1.37.0/networking/httpd.c \
  > analysis_results/framac_httpd.txt 2>&1
```

Justificación de parámetros:

- `-eva`: activa el plugin de análisis de valores, especializado en detectar accesos fuera de rango, uso de variables no inicializadas y posibles *undefined behavior*.
- `-rte`: añade comprobaciones de ejecución (runtime errors) como desbordamientos aritméticos o punteros nulos.
- `-cpp-command`: asegura un preprocesado con GCC usando los mismos includes/defines que BusyBox.

Resultados destacados:

- Varias advertencias de accesos potenciales fuera de límites en buffers (**CWE-119**/**CWE-120**).

- Uso de variables sin inicializar en rutas críticas de código (**CWE-457**).
- Advertencias sobre condiciones de carrera en llamadas a `access()` seguidas de operaciones de archivo (**CWE-362**/**CWE-367**).
- Posibles problemas de punteros nulos al manipular estructuras dinámicas (**CWE-476**).

Comentario: Frama-C es más "exigente" que otras herramientas: su reporte es más ruidoso y formalista, pero permite detectar patrones de riesgo difíciles de ver con Flawfinder o Cppcheck. En este proyecto fue valioso porque confirmó problemas en inicialización de variables y acceso a memoria, además de advertencias de TOCTOU (*time-of-check vs time-of-use*), algo que otras herramientas pasaron por alto.

3.5. Tabla: Análisis de Hallazgos sobre `httpd.c` (2878 LoC) con LLMs

Para la etapa de post-análisis se utilizaron modelos de lenguaje (LLMs) con el fin de validar los resultados obtenidos por las herramientas automáticas. En particular, se empleó **Vertex AI Gemini** para revisar y clasificar los hallazgos de **Flawfinder** y **Cppcheck**, confirmando cuáles correspondían a verdaderos positivos, falsos positivos o posibles falsos negativos. Asimismo, se recurrió a **chat GPT** para apoyar el análisis de los reportes de **Clang Static Analyzer** y **Frama-C**, proporcionando contexto adicional en torno a las advertencias y sugiriendo casos en los que las herramientas pudieron omitir vulnerabilidades lógicas. Esta combinación de herramientas tradicionales con LLMs permitió un análisis más preciso y crítico de las debilidades del código.

Línea	**Herramienta**	**CWE**	**Clasificación**	**Explicación**
315	Flawfinder	CWE-134	VP	`fprintf` con cadena de formato potencialmente influenciada externamente puede derivar en "format string".
725	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito al construir ruta de configuración.
920	Flawfinder	CWE-120	VP	`strcpy` copia sin verificar tamaño del destino.
952	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
960	Flawfinder	CWE-120	VP	`strcpy` sin verificación de tamaño del destino.

Línea	Herramienta	CWE	Clasificación	Explicación
1120	Flawfinder	CWE-120	VP	Construcción de cabeceras en `iobuf` con `sprintf`.
1134	Flawfinder	CWE-120	VP	Concatenación adicional en `iobuf` con `sprintf`.
1168	Flawfinder	CWE-362/367	VP	Posible TOCTOU: `access()` seguido de operación de archivo.
1268	Flawfinder	CWE-120	VP	Más concatenaciones con `sprintf` en `iobuf`.
1292	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1298	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1310	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1317	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1326	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1345	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1349	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1356	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1361	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1370	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1380	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1389	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1401	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1407	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1414	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1419	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1426	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1434	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1440	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1451	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1461	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1471	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1480	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1487	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1495	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1502	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1510	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1518	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1524	Flawfinder	CWE-120	VP	`sprintf` sin límite explícito.
1709	Flawfinder	CWE-78	FP	`execv` se marcó por ejecución de programa; al no invocar shell, no hay interpretación de metacaracteres (riesgo clásico de inyección reducido).
2333	Flawfinder	CWE-120	VP	`strcpy` de URL sin límite explícito.

Línea	Herramienta	CWE	Clasificación	Explicación
2420	Flawfinder	CWE-120	VP	`strcpy` sin verificación de espacio remanente.
2448	Flawfinder	CWE-362/367	VP	Posible TOCTOU con `access`.
2876	Flawfinder	CWE-807	VP	`getenv` puede influir decisiones de seguridad si el entorno es controlable.
856	Flawfinder	CWE-190	VP	`atoi` puede truncar valores grandes silenciosamente.
1645	Flawfinder	CWE-362	VP	Uso de `vfork` con implicaciones de concurrencia en ciertos escenarios.
1658	Cppcheck	CWE-457	VP	`toCgi.wr`/`fromCgi.wr` usados antes de inicializarse (comportamiento indefinido).
1670	Cppcheck	CWE-457	VP	Uso de `toCgi.wr` antes de inicialización.
1673	Cppcheck	CWE-457	VP	Uso de `fromCgi.wr` antes de inicialización.
1911	Cppcheck	CWE-758	VP	Desplazamiento con signo de 64 bits por 63 (UB).
1950	Cppcheck	CWE-758	VP	Desplazamiento con signo por 63 (UB).
2825	Cppcheck	CWE-457	VP	`server_socket` potencialmente usado sin inicializar.
1348	Cppcheck	CWE-398	FP	Alcance de variable `count` (estilo).
1604	Cppcheck	CWE-398	FP	`fullpath` como puntero `const` (estilo).
1699	Cppcheck	CWE-398	FP	`name` como referencia constante (estilo).
1732	Cppcheck	CWE-398	FP	`path` como referencia constante (estilo).
1741	Cppcheck	CWE-398	FP	`path` como referencia constante (estilo).
1757	Cppcheck	CWE-398	FP	`file` como referencia constante (estilo).
1802	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
1822	Cppcheck	CWE-398	FP	`path` como referencia constante (estilo).
1828	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
1833	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
1856	Cppcheck	CWE-398	FP	`range` como referencia constante (estilo).
1885	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
1926	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
1990	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2053	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2094	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2181	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2186	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
2264	Cppcheck	CWE-398	FP	`path` como referencia constante (estilo).
2269	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2295	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
2300	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2310	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).

Línea	Herramienta	CWE	Clasificación	Explicación
2328	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
2371	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
2376	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2390	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2450	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
2455	Cppcheck	CWE-398	FP	`url` como referencia constante (estilo).
2563	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
2570	Cppcheck	CWE-398	FP	`header` como referencia constante (estilo).
1120	Clang SA	CWE-120/242	VP	Acumulación en `iobuf` con `sprintf`; riesgo si `len` supera `IOBUF_SIZE` con entradas largas (cabeceras/URL).
1134	Clang SA	CWE-120/242	VP	Concatenación sucesiva en `iobuf` con `sprintf`; revisar peor caso de suma de longitudes.
1225	Clang SA	CWE-120/242	VP	Misma familia de riesgo por acumulación en `iobuf`.
1263	Clang SA	CWE-242	FP	Literal fijo con `sprintf`; el overflow sólo vendría por acumulación global previa.
1268	Clang SA	CWE-120/242	VP	Cabeceras dinámicas con `sprintf` (acumulación en `iobuf`).
2333	Clang SA	CWE-120/242	Needs-Review	`strcpy(urlcopy, urlp)`; confirmar que `urlcopy` se dimensiona por `strlen(urlp)+1`.
2420	Clang SA	CWE-120/242	Needs-Review	`strcpy(urlp, index_page)`; verificar espacio remanente.
2776	Clang SA	CWE-457	FP-likely	Patrón BusyBox de "silenciar" el compilador; potencial no-inicializada señalada por el checker.
1153	Frama-C (EVA)	CWE-120	VP	Comentario del propio código indica "overflow IS possible" si se usa `sprintf`; EVA avisa por riesgo de fuera de límites en acumulación.
1120	Frama-C (EVA)	CWE-120	VP	Escrituras en `iobuf` dependientes de entrada; potencial *out-of-bounds* si `len` no se acota.
1134	Frama-C (EVA)	CWE-120	VP	Concatenación `iobuf+len` con tamaño no verificado; alerta por acceso fuera de límites.
1225	Frama-C (EVA)	CWE-120	VP	Continuación de concatenaciones; alerta de *out-of-bounds* acumulativo.
1268	Frama-C (EVA)	CWE-120	VP	Cabeceras dinámicas; misma familia de riesgo.

Línea	Herramienta	CWE	Clasificación	Explicación
725	Frama-C (EVA)	CWE-120	Needs-Review	Construcción de filename con `sprintf`; revisar límites reales de destino.
920	Frama-C (EVA)	CWE-120	Needs-Review	Copia desde buffer de config con `strcpy`; confirmar reserva acorde a longitud.
2333	Frama-C (EVA)	CWE-120	Needs-Review	`strcpy(urlcopy, urlp)`; confirmar dimensionamiento exacto del destino.
2420	Frama-C (EVA)	CWE-120	Needs-Review	`strcpy(urlp, index_page)`; validar espacio remanente.

3.6. Verificación de Posibles Falsos Negativos (FN) sobre `httpd.c`

1. Uso de funciones de *string* inseguras (`strncpy`, `strcat`)

Análisis del Código:

- No se utiliza `strcat`. Buena práctica.
- No se usa `strncpy` directamente; aparece `safe_strncpy` de `libbb`, que garantiza terminación NUL.
- Ejemplo:

```
prequest = alloca(16);
safe_strncpy((char*)prequest, iobuf, 16);
```

Conclusión: No es FN. Se prefieren envoltorios seguros de `libbb`.

2. Validación de tamaño en entradas HTTP (Headers, URL, POST)

Análisis del Código:

- **Lectura de headers:** `get_line()` usa `IOBUF_SIZE` (8192) y limita escritura.
- **Tamaño total de headers:** se controla con `MAX_HTTP_HEADERS_SIZE` (32KB).
- **URL:** se copia a `urlcopy` con longitud previamente truncada; sanitización explícita de ...

- **POST:** `Content-Length` con `bb_strtou/BB_STRTOOFF` y validaciones explícitas.

Conclusión: Riesgo bajo/mitigado; los VP de `sprintf/strcpy` se mantienen por prudencia.

3. Manejo de punteros nulos (después de `malloc`)

Análisis del Código:

- Se usan funciones `x...` de `libbb` (`xmalloc`, `xzalloc`, `xstrdup`, `xasprintf`) que abortan ante fallo.
- Además, hay comprobaciones explícitas de nulos en rutas de control (p.ej., `if (urlp == NULL)`; `if (tptr == NULL)`), lo cual reduce FN de dereferencias nulas reportables. :contentReference[oaicite:0]index=0

Conclusión: No es FN. Se elimina gran parte de la clase CWE-476 con `x...` y *guards*.

4. Potencial de desbordamiento de enteros

Análisis del Código:

- Conversión robusta de `Content-Length` y `Range` con `bb_strtou/BB_STRTOOFF`.

Conclusión: Riesgo bajo/mitigado.

5. Uso de `system()` o `popen()` (Command Injection)

Análisis del Código:

- No hay `system()` ni `popen()`.
- `execv(argv[0], argv);` no invoca shell, por lo que no interpreta metacaracteres.

Conclusión: El hallazgo de Flawfinder en la línea 1709 se considera FP respecto a inyección de comandos clásica.

6. Acumulación de `len` en `iobuf` con `sprintf` encadenados

Análisis del Código (Clang + lectura manual):

- Se usan varias llamadas `sprintf(iobuf + len, ...)` para construir cabeceras. Ejemplos explícitos: líneas `~l120`, `~l134`, `~l225`, `~l268`. El propio código deja un comentario: *"Here, overflow IS possible, can't use sprintf:"* en `~l153`. :contentReference[oaicite:1]index=1 :contentReference[oaicite:2]index=2
- Clang SA prioriza estos puntos como *Needs-Review* cuando el tamaño final depende de entradas controlables (query, headers).

Conclusión: FN probable si no se valida `len` global. Sustituir por `snprintf` + comprobación de retorno o limitar la suma acumulada.

7. Copias con `strcpy` condicionadas al dimensionamiento previo

Análisis del Código (Clang + lectura manual):

- `strcpy(cur->before_colon, buf)` (parser de `httpd.conf`) y `strcpy(urlcopy, urlp)` / `strcpy(urlp, index_page)`. :contentReference[oaicite:3]index=3 :contentReference[oaicite:4]index=4
- Si el destino se reserva por el tamaño exacto (p.ej., vía cálculo previo con `strlen`), el riesgo baja. Si hay transformaciones intermedias que aumentan longitud, vuelve a ser riesgo real.

Conclusión: FN posible, pero *FP-likely* si se confirma que el destino proviene de `strlen`+1 exacto. Requiere verificación de rutas de asignación.

3.7. Discusión sobre el análisis estático

El análisis estático acelera muchísimo el hallazgo de patrones de riesgo en C, pero (como vimos) hay que interpretarlo con lupa. En este proyecto el "combo" **Flawfinder + Cppcheck + Clang SA + Frama-C (EVA+RTE)** fue útil y complementario:

- **Flawfinder** hizo un buen barrido de `sprintf`/`strcpy` y marcó `execv`; aquí toca separar el ruido del contexto real (p.ej., `execv` sin shell).

- **Cppcheck** aportó señales de **variables no inicializadas** (p.ej., `server_socket`) y *UB* por shifts de 64 bits: perfecto para endurecer rutas frágiles sin cambiar funcionalidad.

- **Clang Static Analyzer** destacó lo que a mí me parece *lo más delicado del módulo*: **concatenaciones repetidas en** `iobuf` con `sprintf` (acumulando `len`) cuando el input proviene del cliente; incluso el propio código advierte el riesgo en `1153`. *Mi opinión*: aquí es donde hay más valor de hardening hoy. :contentReference[oaicite:5]index=5 :contentReference[oaicite:6]index=6

- **Frama-C (EVA)** no llegó a completar un "pass" formal por el fallo de preprocesado en una primera corrida, pero su *pipeline* sí nos dejó **métrica contextual** (uso intensivo de buffers, funciones inseguras, nulos) y, sobre todo, confirmó que hay *guards* de nulos en varias ramas (`if (urlp == NULL)`, `if (tptr == NULL)`), lo que baja el riesgo de FN por dereferencias. *Interpretación*: aunque no levantó una traza completa, sí aportó evidencia de defensas presentes en el código. :contentReference[oaicite:7]index=7

En resumen: las herramientas no sustituyen el criterio. El punto fuerte aquí es priorizar **iobuf** + `len` (riesgo real si crece más que `IOBUF_SIZE`) y confirmar los `strcpy` "dimensionados". El resto (no inits/UB/logs) son *quick wins* que mejoran robustez sin cambiar la semántica.

3.8. Recomendaciones (basadas en la experiencia)

- **Blindaje de `iobuf`:** sustituir la cadena de `sprintf(iobuf + len, ...)` por `snprintf` verificando **cada retorno** y abortando/recortando si `len` se acerca a `IOBUF_SIZE`. Donde sea viable, construir cabeceras en un *builder* que controle capacidad total. (Clang marcó estos puntos y el propio código advierte overflow). :contentReference[oaicite:8]index=8 :contentReference[oaicite:9]index=9

- **Copias seguras en parser/URL:** reemplazar `strcpy` por `memcpy/strncpy` con tamaño calculado y `NUL` explícito, o mantener `strcpy` solo si el destino proviene de `strlen(src)+1` sin transformaciones posteriores. Verificar `920`, `2333`, `2420`. :contentReference[oaicite:10]index=10 :contentReference[oaicite:11]index=11

- **Inicialización defensiva:** resolver avisos de **use-before-init** (p.ej., `server_socket`) con asignaciones claras al declarar. Son cambios de bajo riesgo que limpian UB e inconsistencias de build.
- **Política de logging:** evitar logs con datos sensibles de peticiones en builds de producción (sanitizar o compilar fuera con `NDEBUG`/flags de release).
- **CI de análisis estático:** integrar `flawfinder`, `cppcheck` y `clang --analyze` en CI con umbrales de fallo (p.ej., # de hallazgos High) y *baselines* por commit.
- **Frama-C en rutas acotadas:** ahora que capturamos los `-I/-D` correctos, correr EVA en funciones críticas (construcción de cabeceras, parser) para intentar demostrar *no-overflow* con `snprintf` y contratos simples (post-condiciones de longitud).
- **Fuzzing ligero:** un *target* que alimente headers/queries largos (y `httpd.conf` "hostiles") para validar que `len` nunca supera el límite y que las copias respetan tamaños.
- **Trazabilidad de CWE:** anotar en el repo una tabla `CSV` con #issue, fichero/línea y CWE consensuado (CWE-120/242/457/532) para mantener consistencia entre herramientas.

Capítulo 4

Referencias

4.1. Enumeración de CWE

1. *ÇWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')"* [Online]. Available: `https://cwe.mitre.org/data/definitions/120.html`

2. *ÇWE-134: Use of Externally-Controlled Format String"* [Online]. Available: `https://cwe.mitre.org/data/definitions/134.html`

3. *ÇWE-190: Integer Overflow or Wraparound"* [Online]. Available: `https://cwe.mitre.org/data/definitions/190.html`

4. *ÇWE-242: Use of Inherently Dangerous Function"* [Online]. Available: `https://cwe.mitre.org/data/definitions/242.html`

5. *ÇWE-362: Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition')"* [Online]. Available: `https://cwe.mitre.org/data/definitions/362.html`

6. *ÇWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition"* [Online]. Available: `https://cwe.mitre.org/data/definitions/367.html`

7. *ÇWE-398: Indicator of Poor Code Quality"* [Online]. Available: `https://cwe.mitre.org/data/definitions/398.html`

8. *ÇWE-457: Use of Uninitialized Variable"* [Online]. Available: `https://cwe.mitre.org/data/definitions/457.html`

9. *ÇWE-532: Insertion of Sensitive Information into Log File"* [Online]. Available: `https://cwe.mitre.org/data/definitions/532.html`

10. *ÇWE-758: Reliance on Undefined, Unspecified, or Implementation-Defined Behavior"* [Online]. Available: `https://cwe.mitre.org/data/definitions/758.html`

11. *ÇWE-807: Reliance on Untrusted Inputs in a Security Decision"* [Online]. Available: `https://cwe.mitre.org/data/definitions/807.html`

4.2. Herramientas de análisis

1. D. A. Wheeler, *"Flawfinder"* [Online]. Available: `https://dwheeler.com/flawfinder/`

2. *Çppcheck - A tool for static C/C++ code analysis"* [Online]. Available: `http://cppcheck.sourceforge.net/`

3. *Çlang Static Analyzer"* [Online]. Available: `https://clang-analyzer.llvm.org/`

4. *"Frama-C - A framework for static analysis of C code"* [Online]. Available: `https://frama-c.com/`